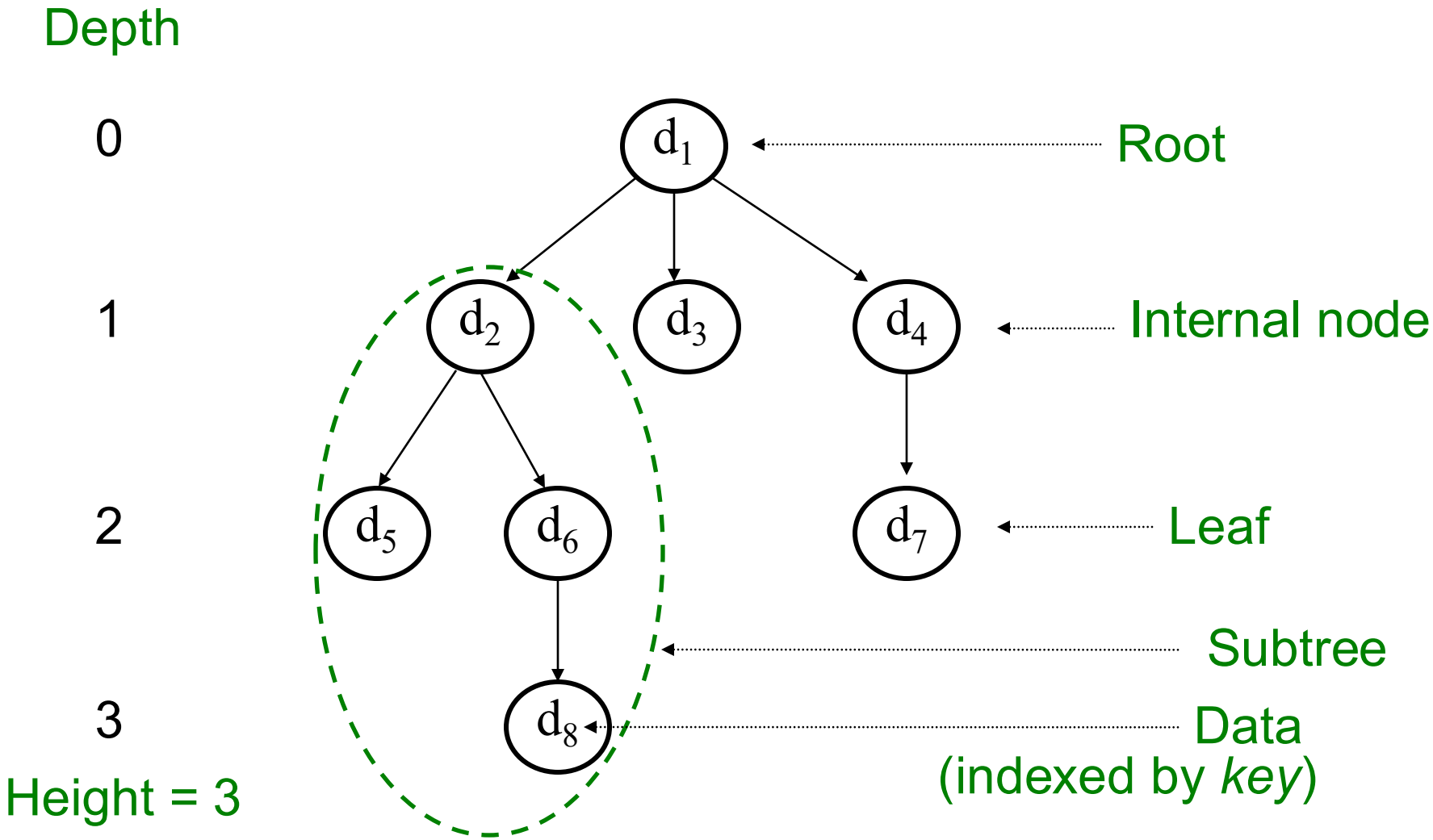
CS3021 Introduction to Data Structures and Intermediate Programming

Binary Search Trees

Objectives

- Learn the properties and key operations for the *Binary Search Tree* (BST).
- Study the efficiency of the BST, and how it compares to other implementations for key-value operations:
  - $O(\log_2 N)$ search
  - $O(1)$ insertion

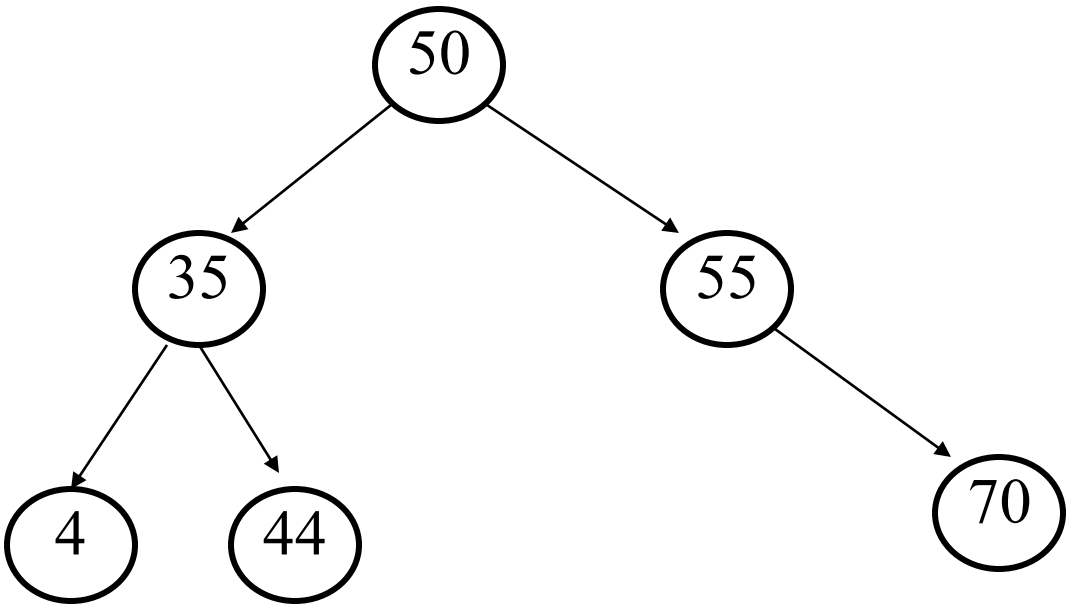
Review: General Tree Structure



Binary Search Trees

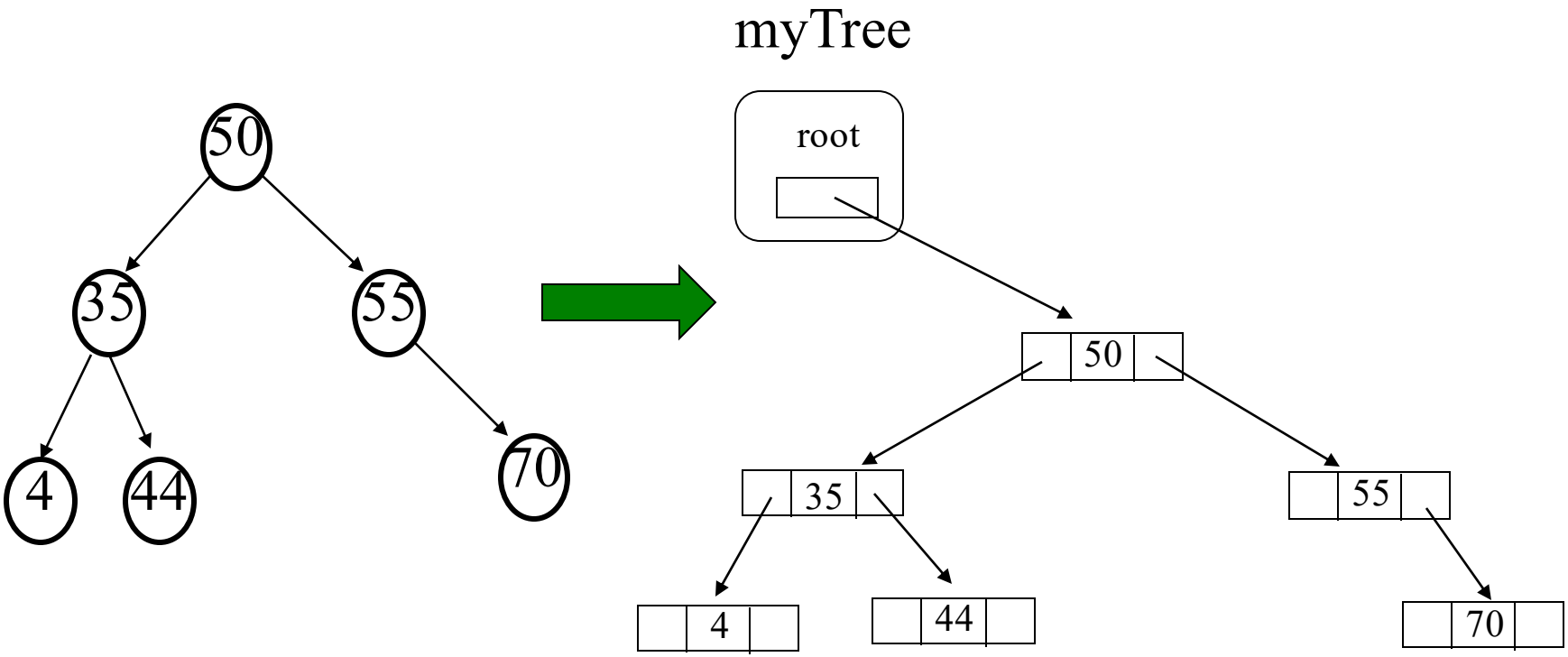
- A *binary search tree* (*BST*) has the properties:
  - it is a binary tree (*what does this mean?*), and
  - for any node *p* in the binary tree:
    - all data in the left *subtree* of *p* has a value less than *p*.data, and
    - all data in the right *subtree* of *p* has a value greater than *p*.data.
- Recursively defined, a *binary tree* is either:
  - an empty tree, or
  - a node that has left and right *subtrees* that are themselves binary trees.

Sample BST



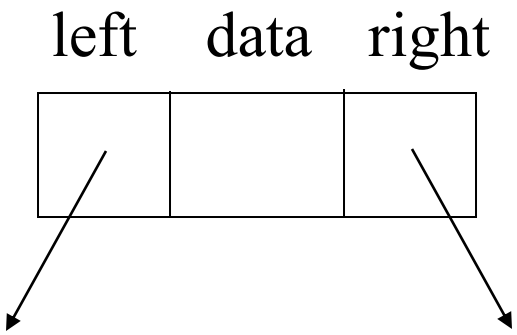
- How does this differ from a *max heap*?

Sample BST Implementation



(Note: blank link fields are nulls)

BST Node Implementation




```
template<class T>
class BinaryTreeNode {
   friend class BinarySearchTree<T>;
private:
   T data;
   BinaryTreeNode *left, *right;
public:
   BinaryTreeNode(BinaryTreeNode *left,
          T data, BinaryTreeNode *right) {
      this->data  = data;
      this->left  = left;
      this->right = right;
   }
   Key getKey() {
      // method to return the key for this node
   }
} // end BinaryTreeNode class
```

BST Implementation (1)

```
class BinarySearchTree {
private:
   BinaryTreeNode *root;
   int count;

public:
   //empty tree constructor
   BinarySearchTree( ): root(0), count(0) {}

   //defined tree constructor
   BinarySearchTree(BinaryTreeNode *root):
         root(root), count(1) {}
```

BST Implementation (2)

```
void insert(BinaryTreeNode *newNode)
{
   //insert a newNode into the BST; the newNode
   // position is determined by its key
}

*BinaryTreeNode search(Key searchKey)
{
   //return a reference to a node which contains
   // an Object whose key equals searchKey
}
```

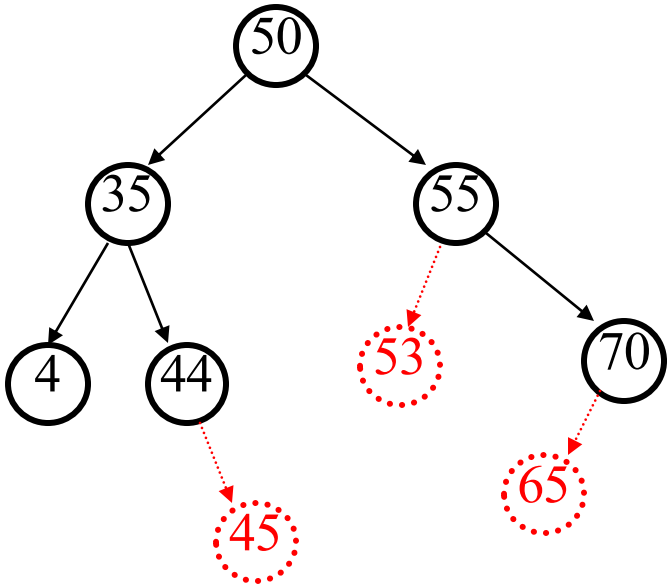
BST Implementation (3)

```
    void delete(Key searchKey)
    {
        //delete a node which contains
        // an Object whose key equals searchKey
    }

    void traverse(int traversalOrder)
    {
        //visit nodes in the tree in traversalOrder
        //Note: Inorder traversal will print out the
        //  values in sorted order.
    }

} //end BinarySearchTree class
```

BST Insert

- A new node *N* is inserted as a leaf to the BST.
- We need a reference to a node that will be a parent of the new node *N*.

- Insert 45
- Insert 65
- Insert 53



BST Insert Algorithm

```
void insert(BinaryTreeNode *newNode)
{
   BinaryTreeNode *parent;
   Key newNodeKey = newNode.getKey();

   parent = findParent(newNodeKey);  //private method

   if (parent == nullptr)
      root = newNode;   //the first node of the BST

   else if (newNodeKey < parent.getKey())
      parent.left = newNode;  //left child of parent

   else
      parent.right = newNode; //right child of parent
} // end insert
```

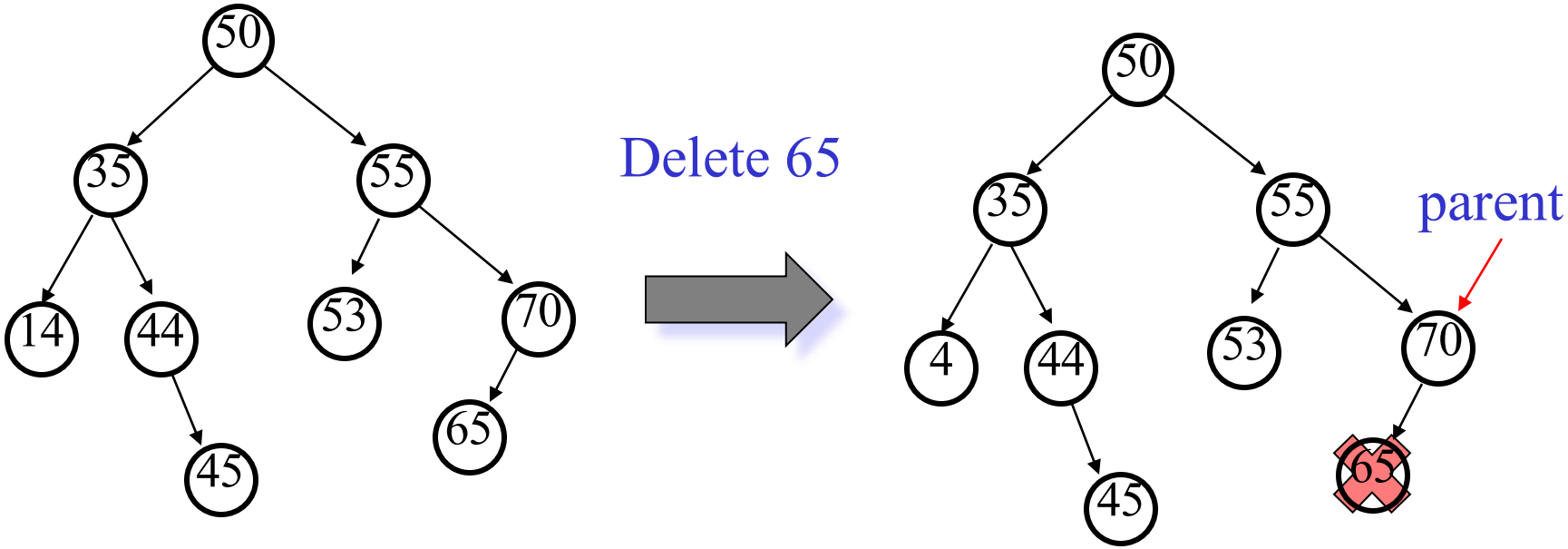
BST Search Algorithm

```
*BinaryTreeNode search(Key searchKey)
{
   BinaryTreeNode *p = root;

   while ((p != nullptr) &&
          (searchKey != p.getKey())) {

      if (searchKey < p.getKey())
         p = p.left;
      else
         p = p.right;
   }

   return p; // null returned if searchKey not found
} // end search
```

BST Delete

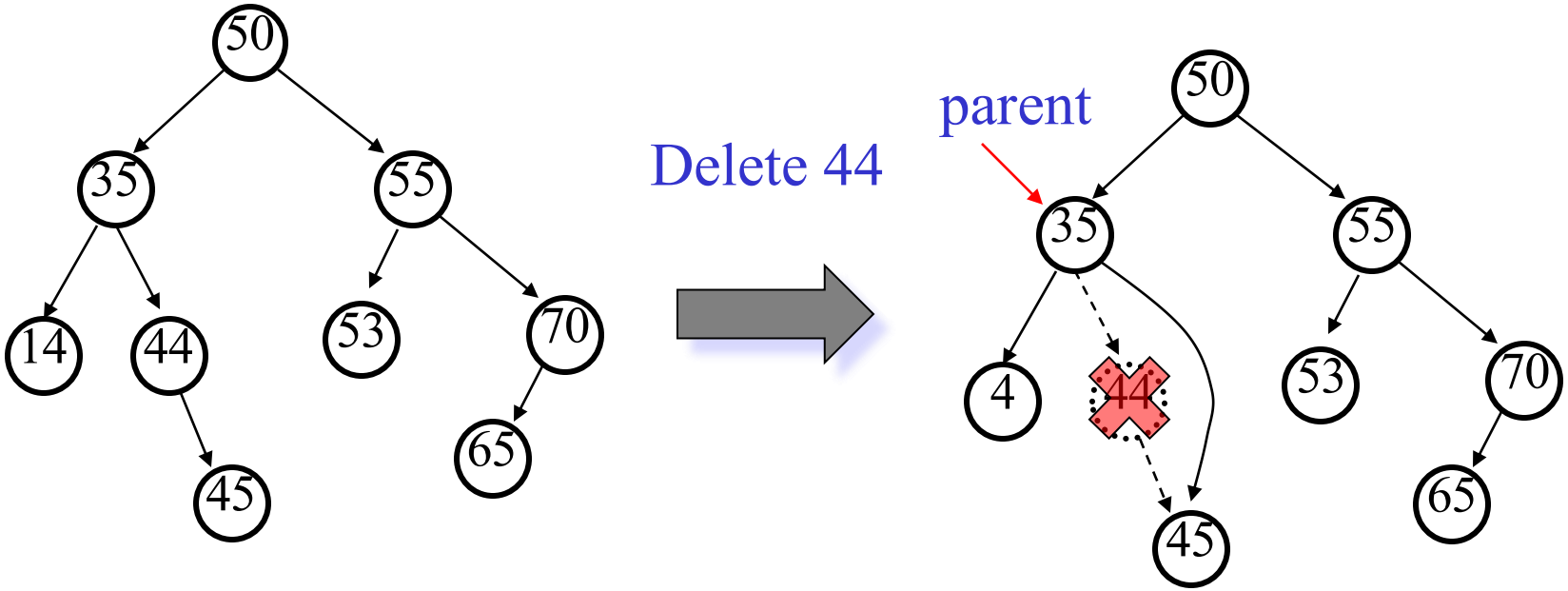
- There are three “cases” to consider when deleting a node from a BST
  - Case 1: Delete a leaf node (trivial)
  - Case 2: Delete a node with one non-empty subtree (still somewhat trivial)
  - Case 3: Delete a node with two non-empty subtrees (requires iteration)

BST Delete: Case 1



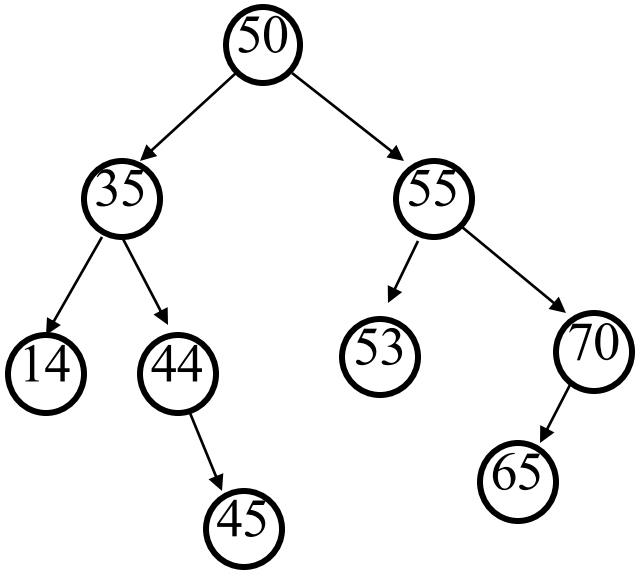
Reset parent.left to null

BST Delete: Case 2, example 1

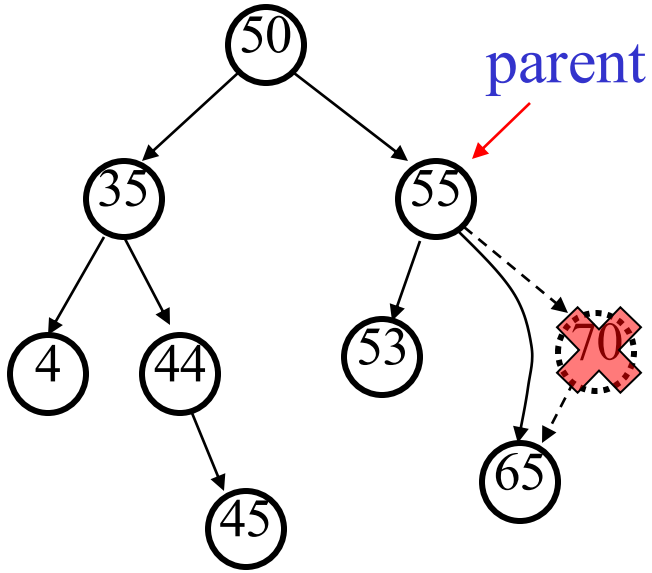


Reset parent.right to parent.right.right

BST Delete: Case 2, example 2

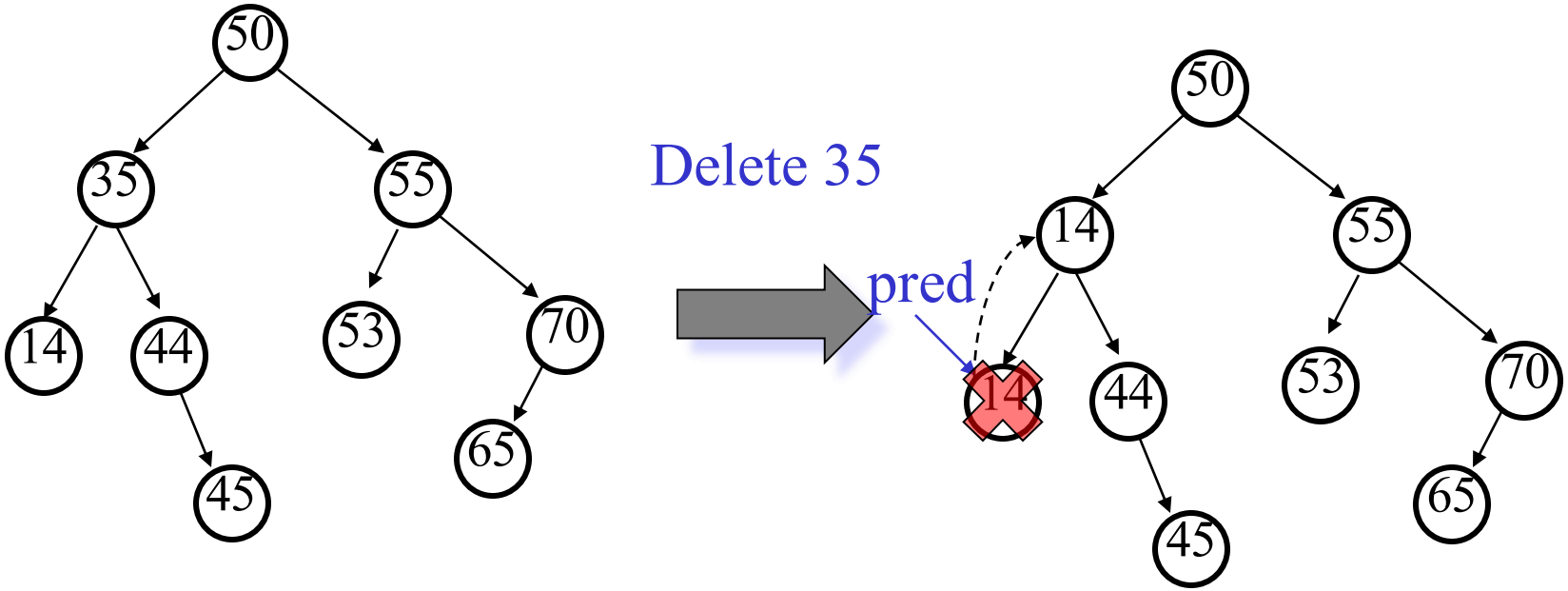


Delete 70



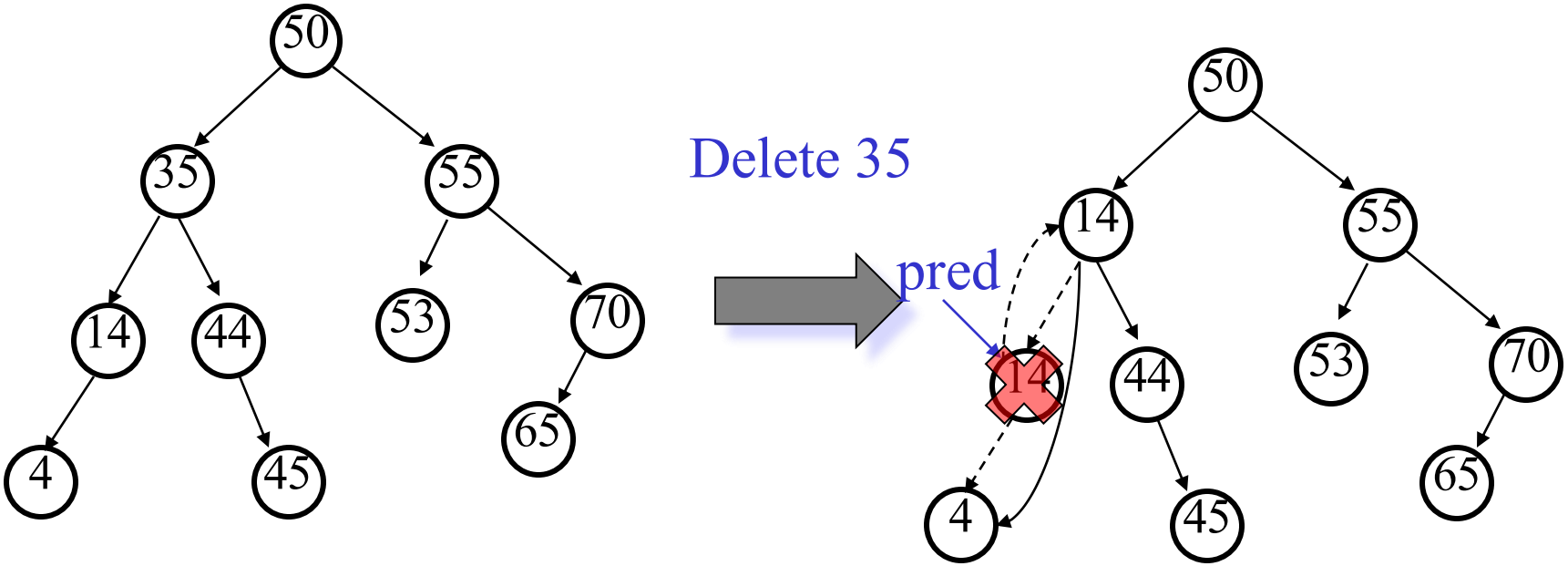
Reset parent.right to parent.right.left

BST Delete: Case 3, example 1



Copy the value of predecessor (14) into delete node, then delete the predecessor node (14, Case 1)

BST Delete: Case 3, example 2



Copy the value of predecessor (14) into delete node, then delete the predecessor node (14, Case 2)

BST Delete Algorithm (1)

```
void delete(Key searchKey) {
    BinaryTreeNode *parent, *delNode;
    delNode = search(searchKey);

    if(delNode != nullptr) {  // is there a node to delete?
        parent = findParent(searchKey);

        if(parent == nullptr &&            // removing root
             delNode.left == nullptr &&    // from 1-node BST
             delNode.right == nullptr)
            root = nullptr;
        else if(delNode.left  != nullptr &&
                delNode.right != nullptr)
            case3Delete(delNode);
        else
            case1or2Delete(parent, delNode);
    }
} // end delete
```

BST Delete Algorithm (2)

```
void case1or2Delete(BinaryTreeNode *parent, *delNode) {
    BinaryTreeNode *child;
    Key delKey = delNode.getKey();

    if(delNode.left == nullptr)
        child = delNode.right;
    else
        child = delNode.left;  // child could be null

    if(delKey < parent.getKey())
        parent.left = child;
    else
        parent.right = child;

} // end case1or2Delete
```

BST Delete Algorithm (3)

```
void case3Delete(BinaryTreeNode *delNode)
{
    BinaryTreeNode *pred, *predParent;

    pred = delNode.left;
    predParent = delNode;

    // find the predecessor
    while(pred.right != nullptr){
        predParent = pred;
        pred       = pred.right;
    }

    delNode.data = pred.data;  // copy predecessor data
    // remove predecessor
    if(delNode.left != pred)
        predParent.right = pred.left;
    else
        predParent.left  = pred.left;
} // end case3Delete
```

BST Traversal

- There are three basic *traversals* through a BST:
  - *Preorder*: N - L - R
  - *Inorder*: L - N - R
  - *Postorder*: L - R - N

(N = node; L = left *subtree* of N; R = right *subtree* of N)

The Traverse Method

```
void traverse(int traversalOrder) {
   if (traversalOrder == PREORDER)
      preorder(root);

   else if (traversalOrder == INORDER)
      inorder(root);

   else  // traversalOrder == POSTORDER
      postorder(root);
} // end traverse
```

Preorder Traversal Algorithm

```
void preorder(BinaryTreeNode *T) {
   if (T != nullptr) {
      visit(T); // perform some operation on T

      preorder(T.left);

      preorder(T.right);
   }
} // end preorder
```

Inorder Traversal Algorithm

```
void inorder(BinaryTreeNode *T) {
   if (T != nullptr) {
      inorder(T.left);

      visit(T); // perform some operation on T

      inorder(T.right);
   }
} // end inorder
```

Postorder Traversal Algorithm

```
void postorder(BinaryTreeNode *T) {
   if (T != nullptr){
      postorder(T.left);

      postorder(T.right);

      visit(T); // perform some operation on T
   }
} // end postorder
```

File I/O with BSTs (1)

- Saving a BST to an external file
  - Recall that a BST can represent data in many ways, but not all are efficient (least efficient representation, O(*N*)).
  - Using *inorder* traversal, this returns node data in key sorted order; instead of printing the node data, we can write BST objects out to an external file.
- Reading in BST data from an external file
  - We can read-in sorted data sequentially from an external file.
  - Use algorithm to build a minimum-height BST (most efficient representation, i.e., $O(\log_2 N)$).

File I/O with BSTs (2)

```
*BinaryTreeNode getFileData(string filename, int n) {
    BinaryTreeNode *treeNode;

    if (n > 0) {
        treeNode = new BinaryTreeNode(nullptr, 0, nullptr);

        // recursively construct left subtree
        treeNode.left = getFileData(filename, n/2);

        // get the root
        // read item from filename into treeNode data

        // recursively construct right subtree
        treeNode.right = getFileData(filename, (n-1)/2);
    }

    return treeNode;

} // end getFileData
```

File I/O with BSTs (3)

- The following method could call getFileData:

```
void readFile( ) {

    // Read number of items stored in file (count)

    // Call method to recursively read data
     BinaryTreeNode *root = getFileData(count);

    // Close buffered reader

} // end readFile
```

Questions?